Industrial Intelligence Agent

A multi-tenant platform that ingests machine sensor data, detects anomalies with PySpark, ranks machines by health risk, and answers “why is M-04 underperforming?” with an evidence-backed investigation — where the facts are computed in code and the language model is only allowed to explain them.

430

TEST FUNCTIONS ACROSS
34 TEST MODULES

3 layers

TENANT ISOLATION — CREDENTIAL, QUERY FILTER, POSTGRES RLS

0

LLM CALLS INSIDE RISK SCORING, DETECTION OR FLEET RANKING

Stack — **FastAPI · Streamlit · LangGraph · PySpark · PostgreSQL 16 (RLS) · Alembic · Gemini / Vertex AI · Pinecone · BM25 · Docker Compose**

> Demonstrated on synthetic data plus the public UCI AI4I 2020 dataset. It has not been deployed at a real factory, and its risk thresholds are calibrated on synthetic data.

Four processes, one trust boundary

The dashboard never opens a database connection and never names a tenant. The token decides the tenant and role; PostgreSQL enforces it. The API and the worker run as separate containers with separate least-privilege database roles.



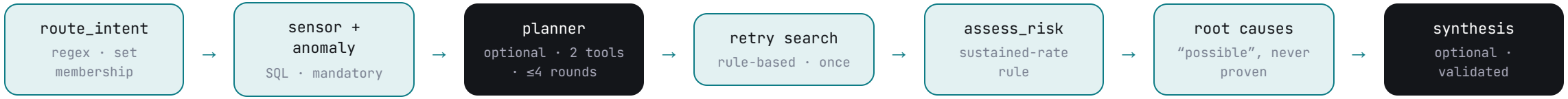
Agent — a LangGraph state graph runs inside the API process, reads PostgreSQL and the tenant's own document index, and returns risk, root-cause candidates, evidence and citations.

No broker — PostgreSQL is the job queue (`FOR UPDATE SKIP LOCKED`), so there is no Redis or message-broker container to run.

Upload storage — a storage-provider abstraction: local volume by default, S3-compatible with one setting.

Facts are computed. Language is optional.

Sensor and anomaly evidence is always gathered deterministically. A Gemini planner may choose only the two *supplementary* sources — maintenance history and document search — and has no tool that could change the risk level.



deterministic code optional LLM step, falls back to deterministic path on any failure

STEP	MECHANISM
Which machine	Parsed from text, reconciled with `machine_id`; unknown → 404, ambiguous → 422
Risk level	`assess_risk` — no model, no database access
Fleet ranking	Same rule over every machine; deterministic template
Narrative	Gemini JSON output checked against the evidence, else template

- **No credentials, no problem** — with no `GOOGLE_CLOUD_PROJECT` the whole agent is deterministic and the test suite runs offline.
- **Every node is traced** — name, machine, duration and failure are logged; recursion is capped by `GRAPH_RECURSION_LIMIT`.

Five decisions that shaped the system

01 A risk rule that doesn't saturate

The first rule — any MEDIUM event means MEDIUM — put **all 18 machines at MEDIUM**, including M-04, whose last 24 hours were fully anomalous. Risk is now the **sustained anomalous-reading rate** over recent hourly windows (≥25% HIGH, ≥5% MEDIUM), with a minimum of six windows.

02 Missing evidence is UNKNOWN, never LOW

A scripted "lazy" planner that skipped every tool made a 100%-anomalous machine read LOW. Sensor and anomaly tools were removed from the planner, and no evidence at all now returns `UNKNOWN`. A regression test replays the lazy planner.

03 A numeric guardrail on the LLM's prose

Every substantive number in the generated report must already exist in the evidence JSON, and the number of cause paragraphs must match the deterministic candidates. Any violation falls back to the template and records why. It checks numbers only, not other claims.

04 Sensors are rows, not columns

Readings are stored long-format (`tenant, machine, sensor, ts, value`), so a new tenant's vocabulary needs a mapping, not a migration. Cost: hourly aggregates still carry only the AI4I columns.

05 Postgres as queue, scheduler and ledger

One database gives at-least-once jobs with idempotent upserts, crash recovery by stale heartbeat, and a connector ledger that prevents double ingestion — at the price of polling latency.

Isolation that fails closed

THREE INDEPENDENT LAYERS

LAYER	WHAT IT ENFORCES
Credential	Tenant comes only from the JWT or API key hash — no request field can name one
Query	Every query takes a required `tenant_id`; a test statically checks that none can skip it
Database	Row-level security with transaction-local tenant context; no tenant bound → zero rows; WITH CHECK blocks cross-tenant writes

ROLES, READ FROM THE DATABASE ON EVERY REQUEST

ROLE	CAN
viewer	Read fleet, investigate, list jobs
admin	+ upload, retry, users, sensor registry, connectors
api_key	Machine client: read, investigate, ingest — no admin

HARDENING

AUTH	**argon2id** passwords, HS256 JWT; one generic 401 for every login failure, one hash verify even for unknown accounts.
ROUTES	Every route is declared in one permission table; a test **fails if a route is missing**.
AUDIT	Logins, denials, role changes and every write recorded with principal, tenant and request id.
SECRETS	Connectors store only a `secret_ref` name; a planted secret is tested never to reach logs, responses or tables.
SSRF	Connector config is bucket / prefix / pattern only — a tenant cannot point the worker at an internal host.
DB	API and worker connect as separate non-owner roles; start-up checks that RLS is actually in effect.

05 — INGESTION & DETECTION

From a raw CSV to a ranked fleet

A tenant uploads an export through the API (or a scheduled GCS pull supplies it). A mapping declares column names and units; a dry run on a sample precedes confirmation. Re-running or retrying never duplicates data.

01
Validate
Every bad row reported with line, column and code; unknown units rejected.

02
Map + normalise
Unit-aware conversion into a long-format schema.

03
Load
Upserts keyed on machine, sensor, timestamp — the idempotency key.

04
Detect
PySpark in the worker: rolling z-score plus registered-range rules.

05
Store
Hourly summaries and anomaly events; job stage updated at each step.

- **Explainable rules** — z-score beyond 3.0 over a trailing 24 h window of 288 readings with the current reading excluded, plus each sensor's registered normal range.
- **Continuity** — detection re-reads a baseline before an upload's first timestamp, so two uploads give the same result as one.
- **Crash safety** — a dead worker is found by stale heartbeat and its job re-queued; transient failures retry with backoff.
- **No double ingestion** — connector ledger is unique on tenant, connector, object key and GCS generation, so two racing schedulers queue a file once.
- **Data used** — an 18-machine synthetic fleet with six injected failure scenarios as ground truth, plus the public AI4I reference set.
- **Second tenant** — a 3-pump "Company B" sample (7,776 readings) runs end to end and stays invisible to the first tenant.

Two backends, one contract

Maintenance documents are chunked once and served by either a dependency-free lexical retriever or a dense one. The agent never goes silent: if cloud credentials are missing, it degrades to BM25.

CORPUS

7 documents

SOPs, equipment manual, safety procedure, troubleshooting guide, incident reports all marked SYNTHETIC

→

CHUNKING

62 chunks

Split on headers, then ~900 chars with 150 overlap; embedding text prefixed with doc id and section

→

LOCAL

BM25

k1 = 1.5, b = 0.75
term-coverage floor 0.3
no network calls

/

CLOUD · OPT-IN

Vertex + Pinecone

gemini-embedding-001
cosine floor 0.75
per-tenant namespace

100%
HIT RATE · 10-CASE BM25 EVALUATION

0.95
MEAN RECIPROCAL RANK ON THE SAME 10 CASES

1
RULE-BASED RETRY WHEN A SEARCH COMES BACK EMPTY

Per tenant
OWN CHUNK FILE OR PINECONE NAMESPACE

> Reading these numbers honestly: ten cases is a small evaluation, two of them accept rank 2 or 3 (BM25 is weaker on paraphrase), and the dense path needs live Vertex AI and Pinecone credentials and is not exercised in CI.

What was proven, and what wasn't

EVIDENCE

- **430 test functions** in 34 modules — RLS, tenant isolation, auth, agent, retrieval, ingestion, connectors, migrations, dashboard (Streamlit AppTest) and an end-to-end demo.
- **Ground truth, not sanity checks** — tests assert against the six injected failure scenarios; the AI4I rule pack reproduces 25,920 summary rows and 28,855 anomaly rows exactly.
- **Full run, Phase 6** (Postgres 16, Python 3.13) — 490 passed, 0 failed, 10 skipped, as recorded in the project's own change log.
- **Operations drills** — a 1.08M-row table queried in ~16 s on index scans; a backup restored with every table's row count matching; `alembic upgrade head` equals `schema.sql`.

KNOWN LIMITS

- **Synthetic data** — thresholds (5% / 25%) are tunable starting points, not validated industrial limits.
- **Not verified** — a clean `docker compose up`, live Vertex / Pinecone runs, and GitHub Actions.
- **Not built** — secrets manager, alerting, retention, password reset, MFA, quotas.
- **Dashboard** — Streamlit, demo-grade; a page refresh signs the user out.
- **Next** — alerting on risk changes, calibration with labelled real data, a retrieval reranker.